Purpose :

This project presents an Exploratory Data Analysis (EDA) of a multi-table banking dataset to evaluate customer behavior, account activity, loan exposure, investment patterns, and risk distribution. The analysis integrates data from Customer, Account, Loan, Transaction, and Investment tables to develop a holistic understanding of the bank's performance and risk profile.Overall, this analysis highlights the importance of customer segmentation, proactive risk management, and targeted product strategies. The insights derived from this EDA can support data-driven decision-making in credit risk assessment, customer retention, and strategic banking initiatives.

In [3]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Collecting Data :

In [4]:
```
file_path = 'Indian_bank_raw_data.xlsx'

customer = pd.read_excel(file_path, sheet_name="Customer Info")
account = pd.read_excel(file_path, sheet_name="Account Info")
loan = pd.read_excel(file_path, sheet_name="Loan Data")
transaction = pd.read_excel(file_path, sheet_name="Transaction Info")
investment = pd.read_excel(file_path, sheet_name="Investments")
```

EDA Starts :

---

1. Customer EDA :

In [5]:
```
customer.head()
```

Out[5]:

	Customer ID	Customer Name	Gender	Age	Bank Name	Occupation	Income	Credit Cards Owened	Properties Owned
0	IND001	Aarav Patil	Male	61.0	SBI	Loan Officer	1250340	3	1.0
1	IND002	Tanvi Deshmukh	F	50.0	Axis Bank	Horticulturist	255348	3	3.0
2	IND003	Siddharth Shinde	m	NaN	SBI	Surgeon	289500	1	NaN
3	IND004	Riya Jadhav	Female	34.0	SBI	Sports Scientist	693996	1	6.0
4	IND005	Aditya Pawar	Male	60.0	SBI	Insurance Inspector	909036	3	1.0

In [6]:

```
customer.isna().sum()
```

Out[6]:

```
Customer ID              0
Customer Name            0
Gender                   0
Age                    123
Bank Name                0
Occupation               0
Income                   0
Credit Cards Owened      0
Properties Owned       101
dtype: int64
```

-----Filling Blank Values-----

In [7]:

```
customer['Age'] = customer['Age'].fillna(customer['Age'].mean()).astype('int64')
customer.head(10)
```

Out[7]:

	Customer ID	Customer Name	Gender	Age	Bank Name	Occupation	Income	Credit Cards Owened	Properties Owned
0	IND001	Aarav Patil	Male	61	SBI	Loan Officer	1250340	3	1.0
1	IND002	Tanvi Deshmukh	F	50	Axis Bank	Horticulturist	255348	3	3.0
2	IND003	Siddharth Shinde	m	45	SBI	Surgeon	289500	1	NaN
3	IND004	Riya Jadhav	Female	34	SBI	Sports Scientist	693996	1	6.0
4	IND005	Aditya Pawar	Male	60	SBI	Insurance Inspector	909036	3	1.0
5	IND006	Shruti Chavan	Female	39	SBI	Systems Analyst	1741368	5	2.0
6	IND007	Rohit More	Male	56	SBI	Administrative Assistant IV	1773660	4	5.0
7	IND008	Prajakta Kadam	Female	57	Axis Bank	Mason	420396	2	4.0
8	IND009	Karan Sawant	Male	55	SBI	Content Writer	457668	5	6.0
9	IND010	Snehal Pawaskar	Female	42	SBI	Dance Instructor	722808	2	6.0

In [8]:

```
customer['Properties Owned'] = customer['Properties Owned'].fillna(customer['Properties Owned'].mean()).astype('int64')
customer.head(10)
```

Out[8]:

	Customer ID	Customer Name	Gender	Age	Bank Name	Occupation	Income	Credit Cards Owened	Properties Owned
0	IND001	Aarav Patil	Male	61	SBI	Loan Officer	1250340	3	1
1	IND002	Tanvi Deshmukh	F	50	Axis Bank	Horticulturist	255348	3	3
2	IND003	Siddharth Shinde	m	45	SBI	Surgeon	289500	1	3
3	IND004	Riya Jadhav	Female	34	SBI	Sports Scientist	693996	1	6
4	IND005	Aditya Pawar	Male	60	SBI	Insurance Inspector	909036	3	1
5	IND006	Shruti Chavan	Female	39	SBI	Systems Analyst	1741368	5	2
6	IND007	Rohit More	Male	56	SBI	Administrative Assistant IV	1773660	4	5
7	IND008	Prajakta Kadam	Female	57	Axis Bank	Mason	420396	2	4
8	IND009	Karan Sawant	Male	55	SBI	Content Writer	457668	5	6
9	IND010	Snehal Pawaskar	Female	42	SBI	Dance Instructor	722808	2	6

-----Replacing Values To Get Consistent Data-----

In [9]:
```
customer['Gender'] = customer['Gender'].replace({'F' : 'Female', 'm' : 'Male', 'mm' : 'Male'})
customer.head(10)
```

Out[9]:

	Customer ID	Customer Name	Gender	Age	Bank Name	Occupation	Income	Credit Cards Owened	Properties Owned
0	IND001	Aarav Patil	Male	61	SBI	Loan Officer	1250340	3	1
1	IND002	Tanvi Deshmukh	Female	50	Axis Bank	Horticulturist	255348	3	3
2	IND003	Siddharth Shinde	Male	45	SBI	Surgeon	289500	1	3
3	IND004	Riya Jadhav	Female	34	SBI	Sports Scientist	693996	1	6
4	IND005	Aditya Pawar	Male	60	SBI	Insurance Inspector	909036	3	1
5	IND006	Shruti Chavan	Female	39	SBI	Systems Analyst	1741368	5	2
6	IND007	Rohit More	Male	56	SBI	Administrative Assistant IV	1773660	4	5
7	IND008	Prajakta Kadam	Female	57	Axis Bank	Mason	420396	2	4
8	IND009	Karan Sawant	Male	55	SBI	Content Writer	457668	5	6
9	IND010	Snehal Pawaskar	Female	42	SBI	Dance Instructor	722808	2	6

-----Creating Groups Based On Age Column-----

In [10]:
```
bins = [19,35,59,100]
labels=['Young Adult', 'Adult', 'Senior']

customer['Age Group'] = pd.cut(customer['Age'], bins=bins, labels=labels, right=True)

customer.head(20)
```

Out[10]:

	Customer ID	Customer Name	Gender	Age	Bank Name	Occupation	Income	Credit Cards Owened	Properties Owned	Age Group
0	IND001	Aarav Patil	Male	61	SBI	Loan Officer	1250340	3	1	Senior
1	IND002	Tanvi Deshmukh	Female	50	Axis Bank	Horticulturist	255348	3	3	Adult
2	IND003	Siddharth Shinde	Male	45	SBI	Surgeon	289500	1	3	Adult
3	IND004	Riya Jadhav	Female	34	SBI	Sports Scientist	693996	1	6	Young Adult
4	IND005	Aditya Pawar	Male	60	SBI	Insurance Inspector	909036	3	1	Senior
5	IND006	Shruti Chavan	Female	39	SBI	Systems Analyst	1741368	5	2	Adult
6	IND007	Rohit More	Male	56	SBI	Administrative Assistant IV	1773660	4	5	Adult
7	IND008	Prajakta Kadam	Female	57	Axis Bank	Mason	420396	2	4	Adult
8	IND009	Karan Sawant	Male	55	SBI	Content Writer	457668	5	6	Adult
9	IND010	Snehal Pawaskar	Female	42	SBI	Dance Instructor	722808	2	6	Adult
10	IND011	Nikhil Bhosale	Male	49	SBI	Horticulturist	290052	2	4	Adult
11	IND012	Manasi Lokhande	Female	30	SBI	Social Media Strategist	934764	1	6	Young Adult
12	IND013	Tejas Naik	Male	29	SBI	Research Assistant I	851520	5	4	Young Adult
13	IND014	Aishwarya Salunkhe	Female	44	SBI	Revenue Officer	1336464	4	6	Adult
14	IND015	Atharva Dalvi	Male	53	SBI	Portfolio Manager	485604	4	3	Adult
15	IND016	Kavya Khot	Female	35	SBI	Security Supervisor	1071624	5	3	Young Adult
16	IND017	Prathamesh Gawande	Male	69	SBI	Housekeeping Staff	1653216	2	1	Senior
17	IND018	Pooja Chorge	Female	30	Axis Bank	Project Manager	1438404	5	4	Young Adult
18	IND019	Harshad Kamble	Male	68	Axis Bank	Gardener	1329540	5	4	Senior

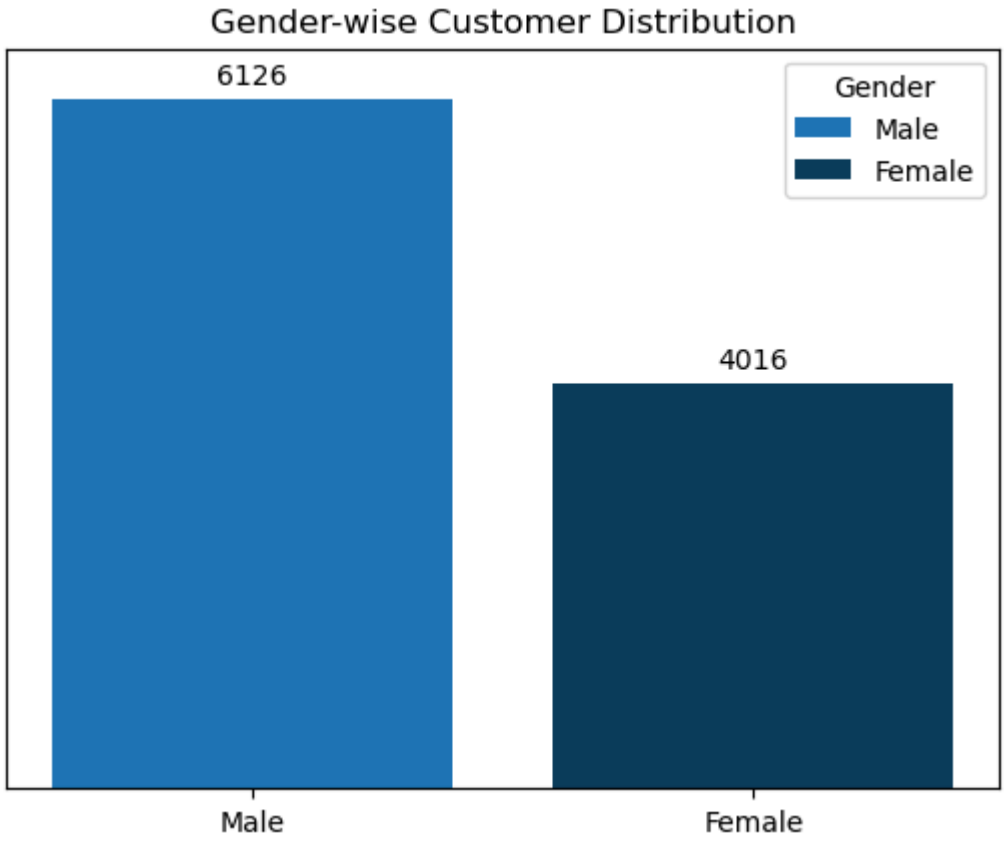
	Customer ID	Customer Name	Gender	Age	Bank Name	Occupation	Income	Credit Cards Owened	Properties Owned	Age Group
19	IND020	Bhagyashree Jagtap	Female	56	Axis Bank	Network Engineer	1291620	4	5	Adult

In [11]:
```
customer.shape
```

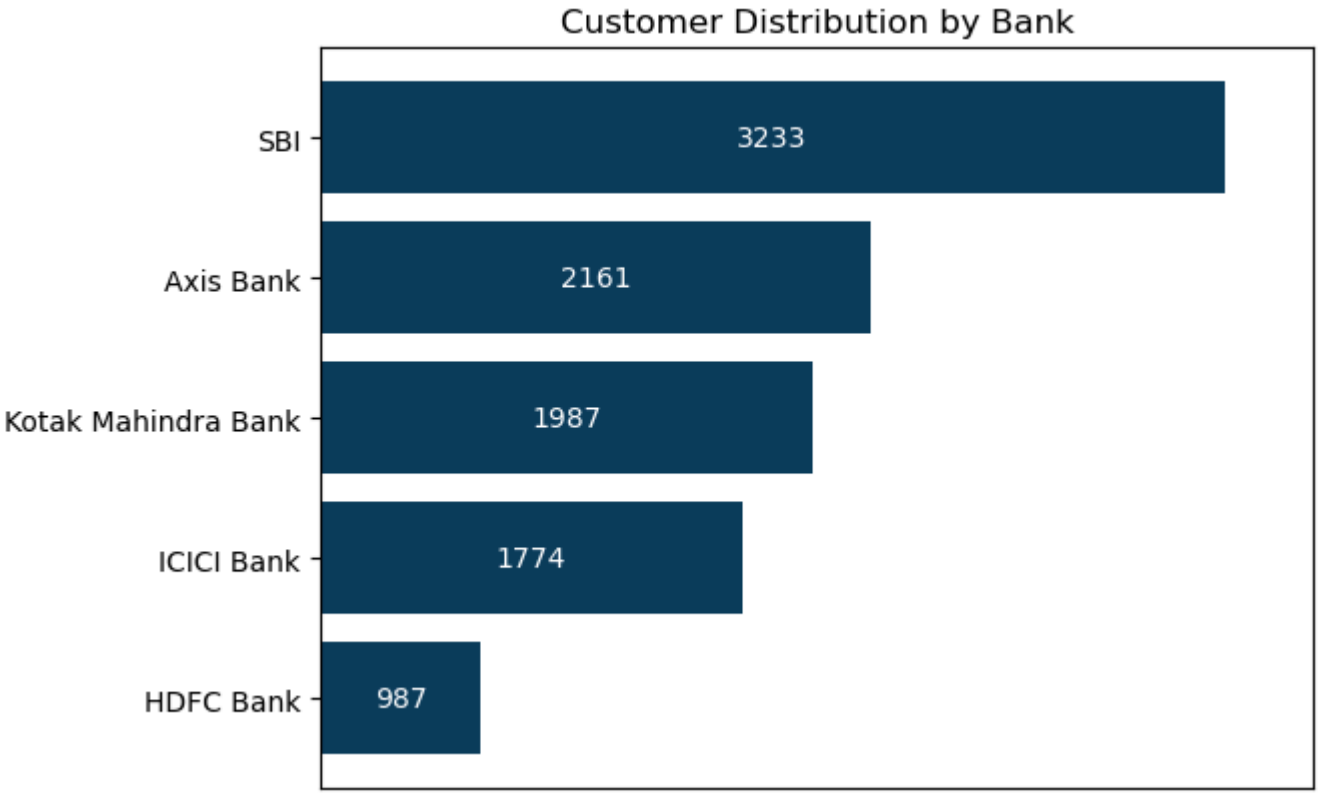
Out[11]:
```
(10142, 10)
```

-----Plotting Diagrams-----

In [12]:
```
# Gender wise Customer

GenXCust = customer.groupby(by=customer['Gender']).agg({'Customer ID' : 'count'}).rename(columns={'Customer ID' : 'Total Customers'}).reset_
GenXCust

colors = ['#1f77b4','#0b3c5d']
bar = plt.bar(x=GenXCust['Gender'], height=GenXCust['Total Customers'], color=colors, label=GenXCust['Gender'])
plt.title('Gender-wise Customer Distribution')
plt.bar_label(bar, labels=GenXCust['Total Customers'], label_type='edge', padding=3, color='black')
plt.legend(title='Gender')
plt.ylim([1000,6500])
plt.gca().get_yaxis().set_visible(False)
plt.show()
```



In [13]:
```
# Bank wise Customer Distribution

CustXBank = customer.groupby(by=customer['Bank Name']).agg({'Customer ID' : 'count'}).rename(columns={'Customer ID' : 'Total Customers'}).s
CustXBank

hbar = plt.barh(CustXBank.index, CustXBank['Total Customers'],  color='#0b3c5d')
plt.bar_label(hbar, labels=CustXBank['Total Customers'], label_type='center', color='w')
plt.title('Customer Distribution by Bank')
plt.gca().get_xaxis().set_visible(False)
plt.xlim([500,3500])
plt.show()
```



In [14]:
```
# Top 10 High Paying Occupations

OccupX10 = customer.groupby(by=customer['Occupation']).agg({'Income' : 'mean'}).round(0).rename(columns={'Income' : 'Average Income'}).asty
OccupX10
```

Out[14]:

	Occupation	Average Income
138	Electronics Engineer	1316878
348	Sales Manager	1280129
149	Film Producer	1262473
152	Financial Analyst	1255088
389	Systems Analyst	1247566
74	Chief Financial Officer	1246079
37	Automation Specialist III	1234650
6	Accountant III	1227798
383	Superintendent of Police	1224010
320	Quality Assurance Engineer	1220756

In [15]:

```
# Average Credit Owned per Customr  #KPI

AvgCreditCard = int(round(customer['Credit Cards Owened'].mean(), 0))
AvgCreditCard
```

Out[15]: 3

In [16]:

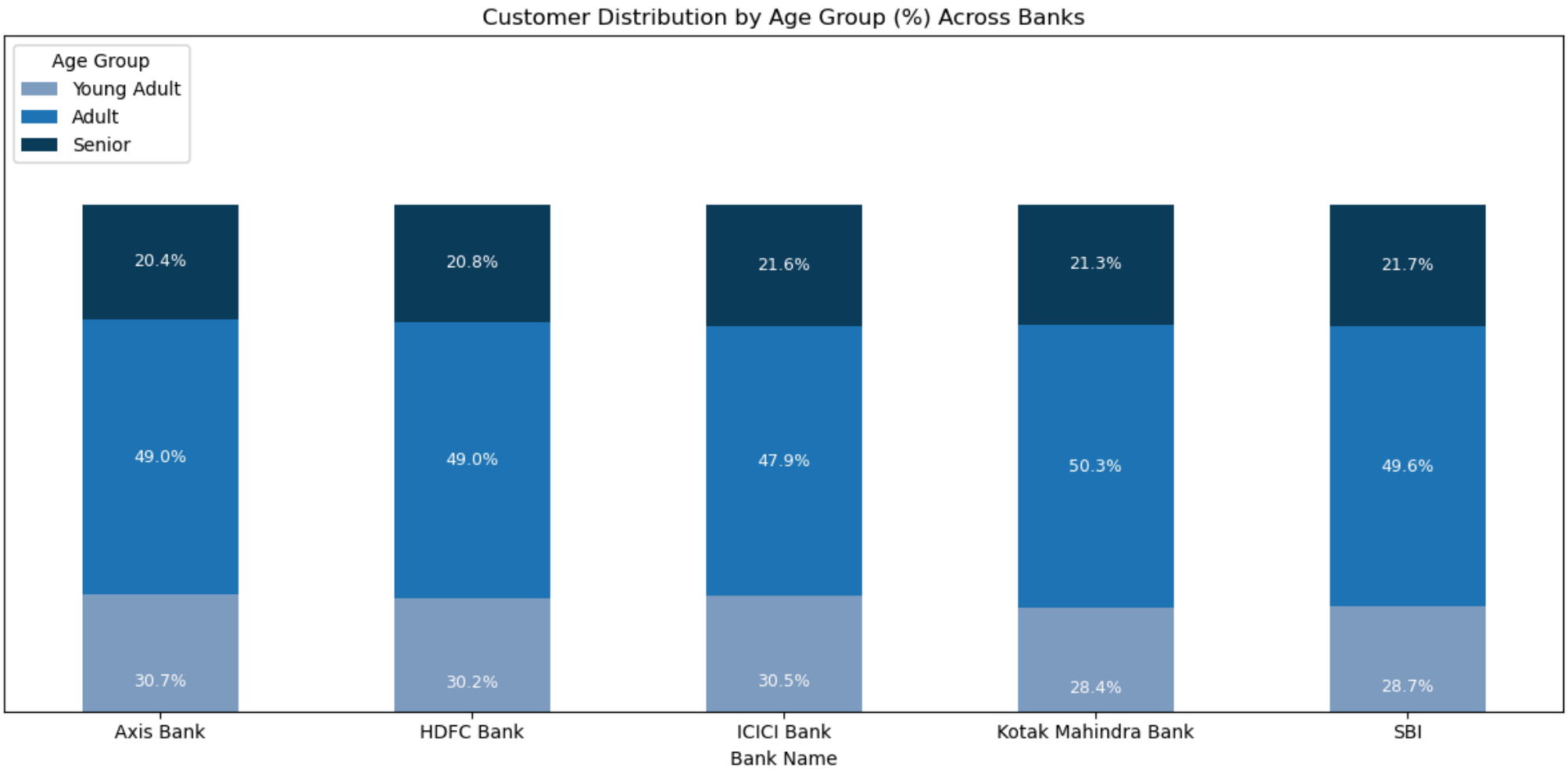
```
# Customer's Average Age  #KPI

AvgCustomerAge = int(round(customer['Age'].mean(), 0))
AvgCustomerAge
```

Out[16]: 45

In [17]:

```
# AgeGroup wise bank preference

GrupXBank = customer.groupby(['Age Group', 'Bank Name'], observed=True).agg({'Customer ID': 'count'}).reset_index().rename(columns={'Custom
GrupXBank

color = ["#7e9bc1", '#1f77b4', '#0b3c5d']

pivot_df = GrupXBank.pivot(index='Bank Name', columns='Age Group', values='Total Customers')
```

```
pivot_pct = pivot_df.div(pivot_df.sum(axis=1), axis=0) * 100

ax = pivot_pct.plot(kind='bar', stacked=True, figsize=(12, 6), color=color)

plt.title('Customer Distribution by Age Group (%) Across Banks')
plt.xticks(rotation=0)
plt.ylim([10,130])
plt.gca().get_yaxis().set_visible(False)
plt.legend(title='Age Group', loc='upper left')

for container in ax.containers:
    for bar in container:
        height = bar.get_height()
        if height >= 5:
            ax.text(bar.get_x() + bar.get_width() / 2,bar.get_y() + height / 2,f"{height:.1f}%",ha='center',va='center',fontsize=9,color='w

plt.tight_layout()
plt.show()
```



In [18]:

```
# Customer's Avg. Income By Age Group

AgeGrupXIncome = customer.groupby(by=['Age Group'], observed=True).agg({'Income' : 'mean'}).round(0).astype('int64').rename(columns={'Incom
AgeGrupXIncome
```

Out[18]:

	Age Group	Avg. Income
0	Young Adult	1005215
1	Adult	995432
2	Senior	998053

Customer Analysis – Key Insights :

The customer base is unevenly distributed across demographic and risk categories, indicating varying financial behaviors and engagement levels.

A concentrated customer segment contributes significantly to overall banking activity, suggesting dependency on specific customer profiles.

A noticeable proportion of customers fall under medium to high-risk categories, highlighting potential exposure to credit and default risk.

Low-risk customers form a stable base, offering opportunities for long-term relationship building and cross-selling.

---

2. Account EDA :

In [19]:
```
account.head()
```

Out[19]:

	Customer ID	Salary	Bank Loan	Bank Loan Status	Business Lending	Business Lending Status	Current Account	Savings Account	Fixed Deposit	Credit Card Balance	Superannuation Savings	Joined Bank	Tendency	Risk Category
0	IND001	104195	323004.50	Closed	1295143.85	Active	5845	117740.35	482044	5590	1028150	2016-09-11	Spender	Low Risk
1	IND002	21279	69582.33	Closed	66816.06	Active	5940	29152.23	389668	13849	48537	2017-02-22	Spender	High Risk
2	IND003	24125	48250.00	Closed	84196.25	Active	12443	13027.50	426355	24919	60210	2002-02-17	Spender	High Risk
3	IND004	57833	131859.24	Closed	793468.76	Active	4426	65929.62	372307	12097	33325	2019-11-17	Spender	Medium Risk
4	IND005	75753	278013.51	Closed	473456.25	Active	13661	42421.68	414732	19240	1132699	2009-11-10	Spender	Medium Risk

In [20]:
```
account.dtypes
```

Out[20]:
```
Customer ID                        object
Salary                              int64
Bank Loan                         float64
Bank Loan Status                   object
Business Lending                  float64
Business Lending Status            object
Current Account                     int64
Savings Account                   float64
Fixed Deposit                       int64
Credit Card Balance                 int64
Superannuation Savings              int64
Joined Bank                 datetime64[ns]
Tendency                           object
Risk Category                      object
dtype: object
```

-----Changing Data Types-----

In [21]:
```
account['Bank Loan'] = account['Bank Loan'].astype('int64')
account['Business Lending'] = account['Business Lending'].astype('int64')
account['Savings Account'] = account['Savings Account'].astype('int64')
```

-----Creating Columns Using Date&Time Function-----

In [22]:
```
account['Joined Year'] = pd.DatetimeIndex(account['Joined Bank']).year
```

In [23]:
```
account['Joined Month'] = pd.DatetimeIndex(account['Joined Bank']).month_name()
```

In [24]:
```
account.head()
```

Out[24]:

	Customer ID	Salary	Bank Loan	Bank Loan Status	Business Lending	Business Lending Status	Current Account	Savings Account	Fixed Deposit	Credit Card Balance	Superannuation Savings	Joined Bank	Tendency	Risk Category	Joined Year	
0	IND001	104195	323004	Closed	1295143	Active	5845	117740	482044	5590	1028150	2016-09-11	Spender	Low Risk	2016	S
1	IND002	21279	69582	Closed	66816	Active	5940	29152	389668	13849	48537	2017-02-22	Spender	High Risk	2017	
2	IND003	24125	48250	Closed	84196	Active	12443	13027	426355	24919	60210	2002-02-17	Spender	High Risk	2002	
3	IND004	57833	131859	Closed	793468	Active	4426	65929	372307	12097	33325	2019-11-17	Spender	Medium Risk	2019	N
4	IND005	75753	278013	Closed	473456	Active	13661	42421	414732	19240	1132699	2009-11-10	Spender	Medium Risk	2009	N

-----Plotting Graphs-----

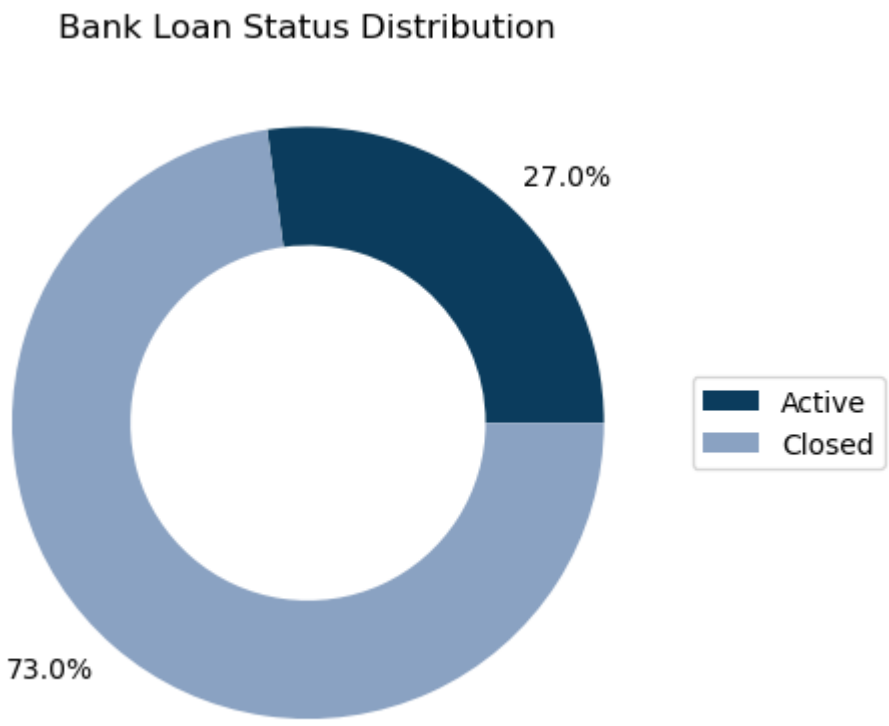
In [25]:

```
# Average Customer Salary  #KPI

AvgSal = int(round(account['Salary'].mean(), 0))
AvgSal
```

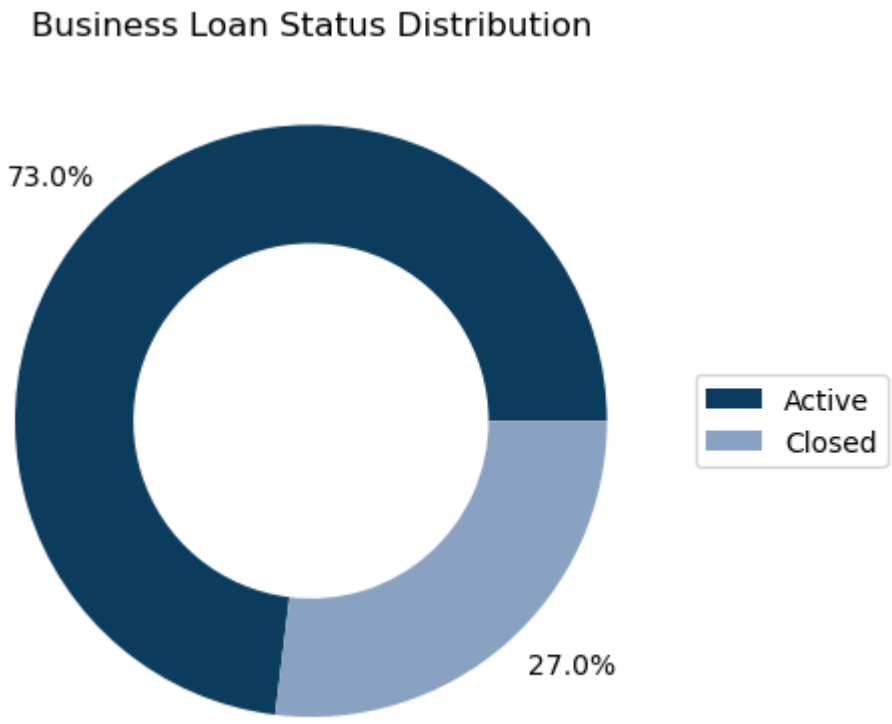
Out[25]: 83240

In [26]:

```
# Bank Loan Status Distribution

BlsXcust = account.groupby(by=['Bank Loan Status']).agg({'Customer ID' : 'count'}).reset_index().rename(columns={'Customer ID' : 'Total Cus
BlsXcust

color = ['#0b3c5d', "#8aa2c2"]
labels = (BlsXcust['Total Customers'] / BlsXcust['Total Customers'].sum() * 100).round(0).astype(str) + '%'

plt.pie(BlsXcust['Total Customers'], labels=labels, wedgeprops={'width' : 0.4}, colors=color)
plt.title('Bank Loan Status Distribution')
plt.legend(BlsXcust['Bank Loan Status'], loc='center left', bbox_to_anchor=(1, 0.5))
plt.show()
```



In [27]:
```
# Business Loan Status Distribution

BusinessXcust = account.groupby(by=['Business Lending Status']).agg({'Customer ID' : 'count'}).rename(columns={'Customer ID' : 'Total Custo
BusinessXcust

label = (BusinessXcust['Total Customers'] / BusinessXcust['Total Customers'].sum() * 100).round(0).astype(str) + '%'
color = ['#0b3c5d', "#8aa2c2"]

plt.pie(BusinessXcust['Total Customers'], labels=label, wedgeprops={'width' : 0.4}, colors=color)
plt.legend(BusinessXcust['Business Lending Status'], loc='center left', bbox_to_anchor=(1, 0.5))
plt.title("Business Loan Status Distribution")
plt.show()
```



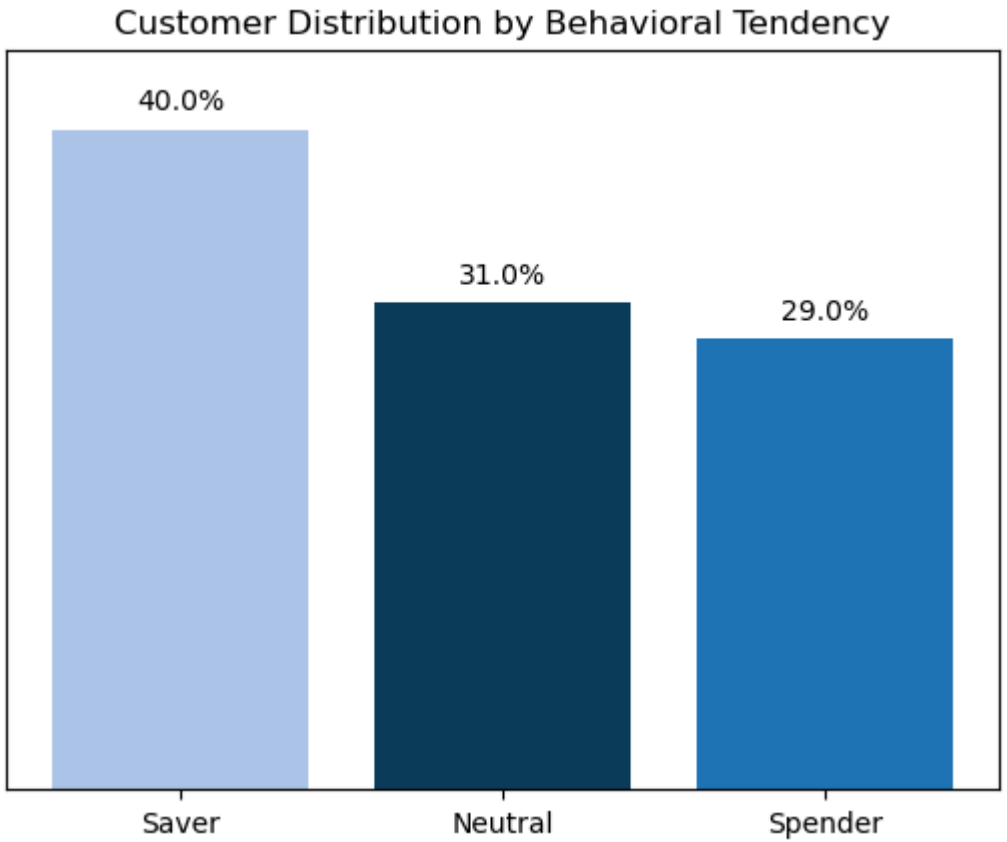
In [28]:
```
# Year Wise Customer Growth

YearXCust = account.groupby(by=['Joined Year']).agg({'Customer ID' : 'count'})
YearXCust
```

Out[28]:

Joined Year	Customer ID
2002	467
2003	414
2004	458
2005	464
2006	417
2007	439
2008	426
2009	424
2010	457
2011	459
2012	388
2013	453
2014	459
2015	448
2016	455
2017	422
2018	432
2019	427
2020	454
2021	458
2022	455
2023	465
2024	401

In [29]:

```
# Customer Tendency

CustXTen = account.groupby(by=['Tendency']).agg({'Customer ID' : 'count'}).rename(columns={'Customer ID' : 'Total Customers'}).reset_index(
CustXTen

label = (CustXTen['Total Customers'] / CustXTen['Total Customers'].sum() * 100).round(0).astype(str) + '%'
color = ['#1f77b4', '#0b3c5d', '#aec7e8']

bar = plt.bar(CustXTen['Tendency'], height=CustXTen['Total Customers'], color=color)
plt.gca().invert_xaxis()
plt.bar_label(bar, labels=label, label_type='edge', padding=5)
plt.ylim([500,4500])
plt.gca().get_yaxis().set_visible(False)
plt.title('Customer Distribution by Behavioral Tendency')
plt.show()
```



In [30]:
```
# Total Business & Bank Loan

TotalXAll = pd.DataFrame(account[['Bank Loan', 'Business Lending']].sum())
TotalXAll
```

Out[30]:

	0
Bank Loan	2118186799
Business Lending	7560358753

In [31]:
```
# Average Credit Card Balance  #KPI

AvgCreditBal = int(round(account['Credit Card Balance'].mean(), 0))
AvgCreditBal
```

Out[31]: 13421

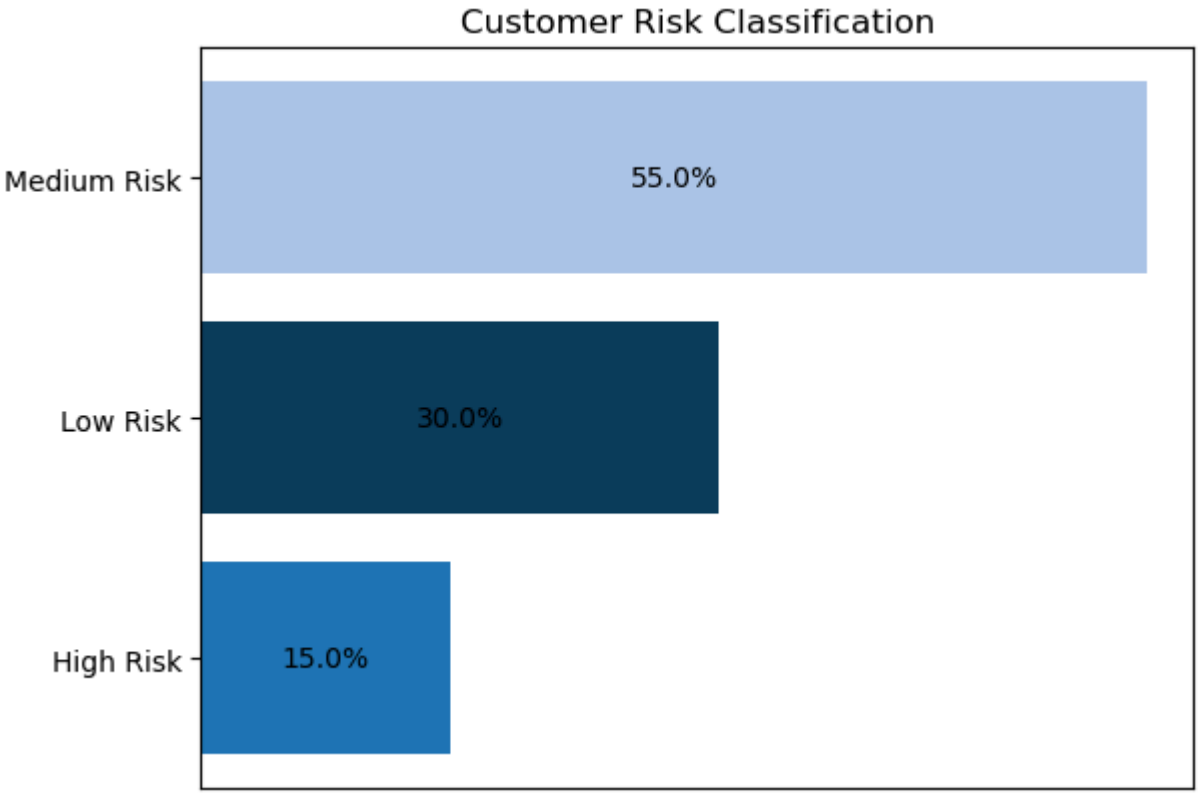
In [32]:
```
account.describe()
```

Out[32]:

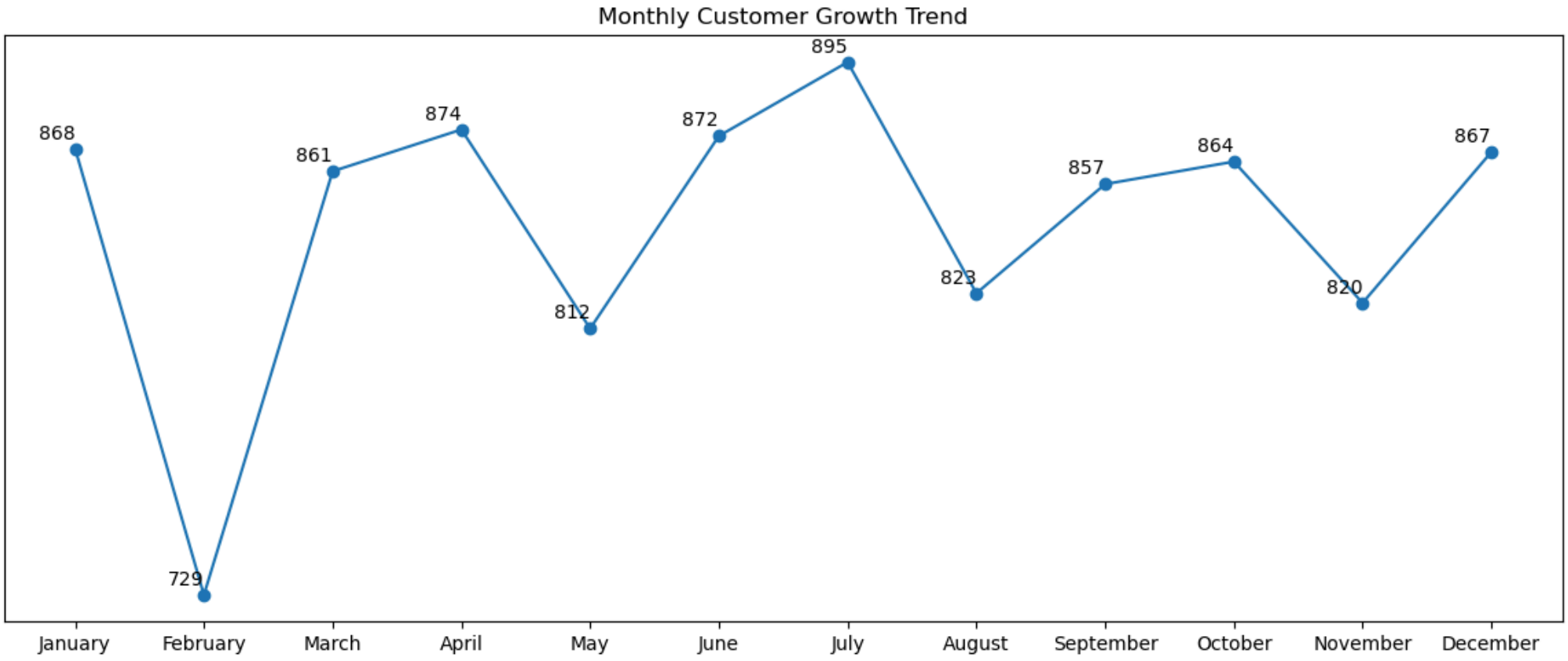
	Salary	Bank Loan	Business Lending	Current Account	Savings Account	Fixed Deposit	Credit Card Balance	Superannuation Savings	Joined Bank	Join
count	10142.00000	10142.000000	1.014200e+04	10142.000000	10142.000000	10142.000000	10142.000000	1.014200e+04	10142	10142.
mean	83239.63025	208852.967758	7.454505e+05	8464.405936	83514.934332	425746.087852	13421.403569	5.026955e+05	2013-06-30 19:30:39.400512256	2012.
min	15003.00000	16184.000000	4.898600e+04	2000.000000	3364.000000	100000.000000	2000.000000	8.007000e+03	2002-01-01 00:00:00	2002.
25%	50043.50000	107233.750000	3.625515e+05	5238.250000	35634.500000	388822.250000	7657.000000	3.489450e+04	2007-09-15 00:00:00	2007.
50%	83615.00000	185036.000000	6.332000e+05	8418.000000	68681.000000	425886.000000	13404.000000	6.100250e+04	2013-07-12 00:00:00	2013.
75%	117305.00000	291936.000000	1.046197e+06	11680.250000	120404.750000	463001.750000	19138.750000	1.209225e+06	2019-04-25 00:00:00	2019.
max	149999.00000	595297.000000	2.235851e+06	15000.000000	268943.000000	499983.000000	25000.000000	1.999907e+06	2024-12-30 00:00:00	2024.
std	38943.81602	126167.399806	4.765745e+05	3753.871470	58365.305069	43150.367207	6625.526826	6.962274e+05	NaN	6.

In [33]:

```
# Customer Risk Category

CustXRisk = account.groupby(by=['Risk Category']).agg({'Customer ID' : 'count'}).rename(columns={'Customer ID'  : 'Total Customers'}).reset_
CustXRisk

labels = (CustXRisk['Total Customers'] / CustXRisk['Total Customers'].sum() * 100).round(0).astype(str) + '%'
color = ['#1f77b4', '#0b3c5d', '#aec7e8']

bar = plt.barh(CustXRisk['Risk Category'], CustXRisk['Total Customers'], color=color)
plt.bar_label(bar, labels=labels, label_type='center')
plt.gca().get_xaxis().set_visible(False)
plt.title('Customer Risk Classification')
plt.show()
```



In [34]:
```
# Monthly Customer Trend

month_order = ["January", "February", "March", "April", "May", "June", "July", "August", "September", "October", "November", "December"]

MonthXCust = account.groupby(by=['Joined Month']).agg({'Customer ID' : 'count'}).reindex(month_order).rename(columns={'Customer ID' : 'Tota
MonthXCust

plt.figure(figsize=[12,5])
plt.plot(MonthXCust['Joined Month'], MonthXCust['Total Customers'], marker='o')
plt.title('Monthly Customer Growth Trend')
plt.tight_layout()
plt.gca().get_yaxis().set_visible(False)

for x, y in zip(MonthXCust['Joined Month'], MonthXCust['Total Customers']):
    plt.text(x, y + 3, str(y), ha='right', fontsize=10)

plt.show()
```



Account Performance – Key Insights :

Customers typically maintain a limited number of accounts, indicating conservative banking behavior rather than aggressive financial diversification.

High-value accounts are concentrated among a smaller customer segment, suggesting unequal distribution of banking assets.

Customers with multiple accounts demonstrate higher engagement levels, making them ideal candidates for targeted financial products and premium services.

---

3. Loan EDA :

In [35]:
```
loan.head()
```

Out[35]:

	Loan ID	Customer ID	Loan Type
0	LN01	IND8491	Personal Loan
1	LN02	IND1780	Personal Loan
2	LN03	IND5731	Personal Loan
3	LN04	IND5389	Personal Loan
4	LN05	IND6570	Personal Loan

In [36]:
```
loan.dtypes
```

Out[36]:
```
Loan ID        object
Customer ID    object
Loan Type      object
dtype: object
```

In [37]:
```
loan.isna().sum()
```

Out[37]:
```
Loan ID         0
Customer ID     0
Loan Type      27
dtype: int64
```

-----Filling Blank Values-----

In [38]:
```
loan['Loan Type'] = loan['Loan Type'].fillna(loan['Loan Type'].mode()[0])
```

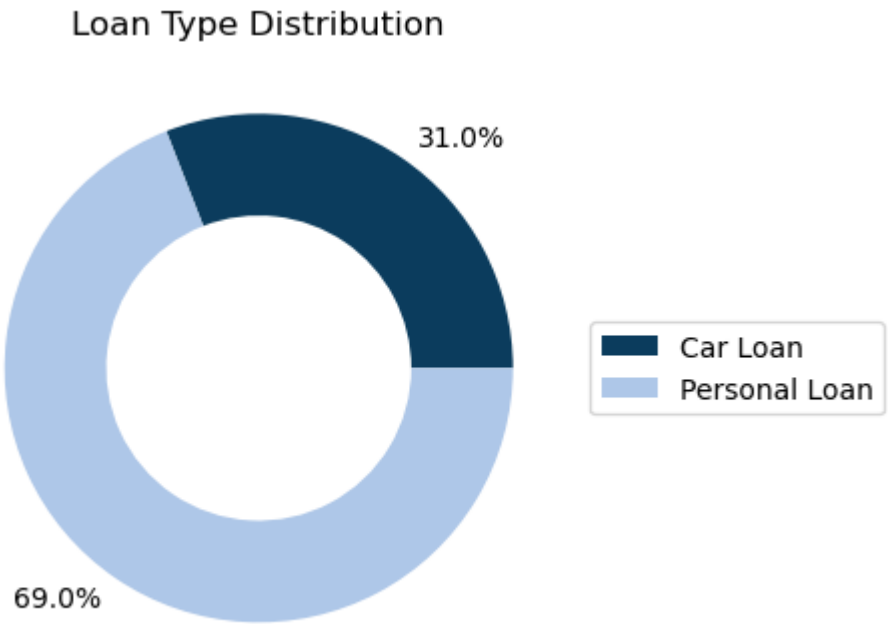
In [39]:
```
loan.isna().sum()
```

Out[39]:
```
Loan ID        0
Customer ID    0
Loan Type      0
dtype: int64
```

-----Plotting Graphs-----

In [40]:
```
# Loan Type Distribution

LoanTypeXCust = loan.groupby(by=['Loan Type']).agg({'Loan ID' : 'count'}).rename(columns={'Loan ID' : 'Total Customers'}).reset_index()
LoanTypeXCust
```

```
label = (LoanTypeXCust['Total Customers'] / LoanTypeXCust['Total Customers'].sum() * 100).round(0).astype(str) + '%'
color = ['#0b3c5d', '#aec7e8']

plt.figure(figsize=(5,5))
plt.pie(LoanTypeXCust['Total Customers'], labels=label, wedgeprops={'width' : 0.4}, colors=color)
plt.legend(LoanTypeXCust['Loan Type'], loc='center left', bbox_to_anchor=(1,0.5))
plt.title('Loan Type Distribution')
plt.tight_layout()
plt.show()
```



Loan Analysis – Key Insights :

Loan exposure is skewed towards specific loan types, reflecting customer preference for certain credit products.

A small subset of customers accounts for a disproportionately high share of total loan amounts.

High-risk customers contribute significantly to overall loan exposure, increasing the bank's vulnerability to defaults.

This pattern emphasizes the need for tighter credit assessment and monitoring mechanisms for high-risk segments.

---

4. Transaction EDA :

In [41]:
```
transaction.head()
```

Out[41]:

	TID	Customer ID	Transaction Type	Transaction Mode	Amount
0	TRXN001	IND9933	Debit	UPI	7910
1	TRXN002	IND9328	Debit	UPI	2436
2	TRXN003	IND9324	Debit	UPI	9876
3	TRXN004	IND704	Debit	UPI	7063
4	TRXN005	IND2268	Debit	UPI	7016

In [42]:
```
transaction.dtypes
```

Out[42]:
```
TID                 object
Customer ID         object
Transaction Type    object
Transaction Mode    object
Amount               int64
dtype: object
```

In [43]:
```
transaction.shape
```

Out[43]:
```
(40000, 5)
```

In [44]:
```
transaction.isna().sum()
```

Out[44]:
```
TID                 0
Customer ID         0
Transaction Type    0
Transaction Mode    0
Amount              0
dtype: int64
```

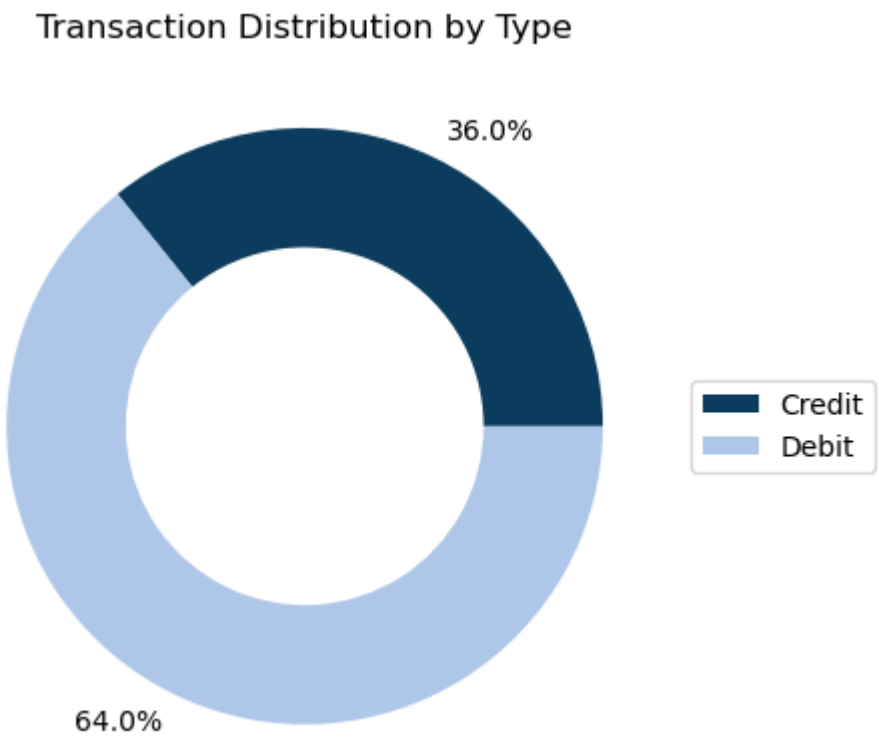
In [45]:
```
# Total Transactions  #KPI
```

```
TotalTransactions = transaction['TID'].count()
TotalTransactions
```

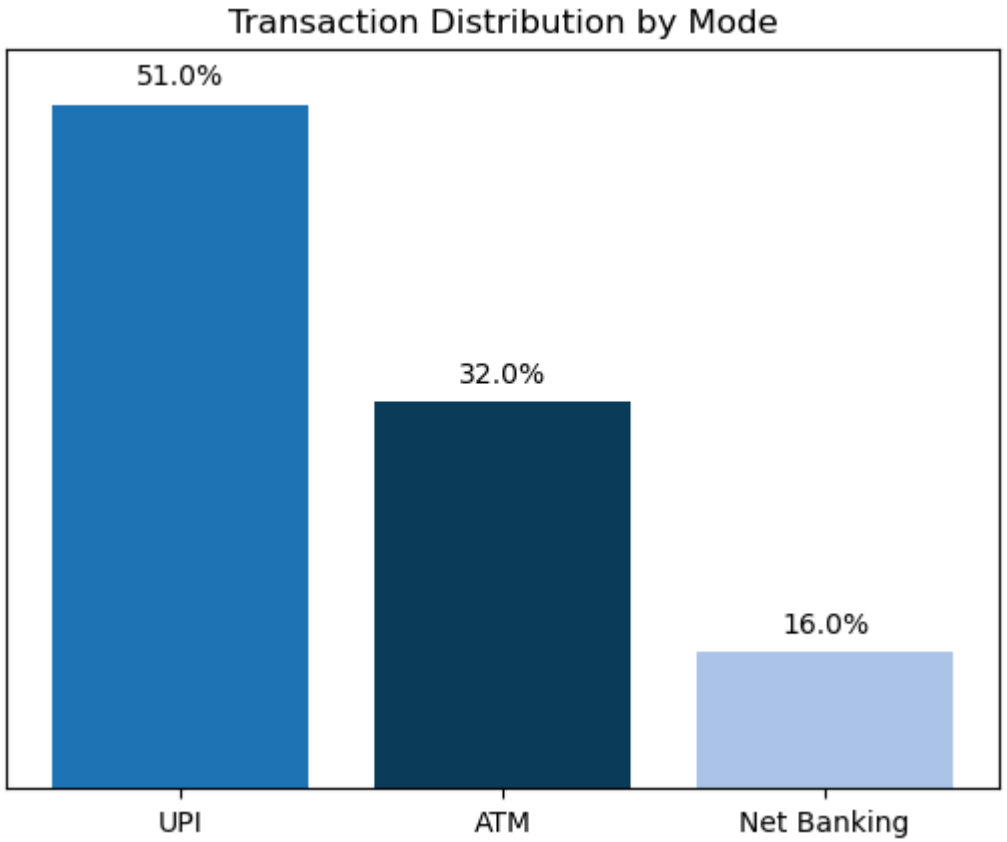
Out[45]: np.int64(40000)

In [46]:

```
# Transactions By Transaction Types

TransXType = transaction.groupby(by=['Transaction Type']).agg({'TID' : 'count'}).rename(columns={'TID' : 'Total Transactions'}).reset_index
TransXType

label = (TransXType['Total Transactions'] / TransXType['Total Transactions'].sum() * 100).round(0).astype(str) + '%'
color = ['#0b3c5d', '#aec7e8']

plt.figure(figsize=(5,5))
plt.pie(TransXType['Total Transactions'], labels=label, wedgeprops={'width' : 0.4}, colors=color)
plt.legend(TransXType['Transaction Type'], loc='center left', bbox_to_anchor=(1, 0.5))
plt.title('Transaction Distribution by Type')
plt.tight_layout()
plt.show()
```



In [47]:
```
# Transactions By Transaction Mode

TransXMode = transaction.groupby(by=['Transaction Mode']).agg({'TID' : 'count'}).rename(columns={'TID' : 'Total Transactions'}).reset_index
TransXMode

label = (TransXMode['Total Transactions'] / TransXMode['Total Transactions'].sum() * 100).round(0).astype(str) + '%'
color = ['#1f77b4', '#0b3c5d', '#aec7e8']

bar = plt.bar(TransXMode['Transaction Mode'], TransXMode['Total Transactions'], color=color)
plt.bar_label(bar, labels=label, label_type='edge', padding=5)
plt.ylim([3000,22000])
plt.title('Transaction Distribution by Mode')
plt.gca().get_yaxis().set_visible(False)
plt.show()
```



In [48]:
```
# Average Transaction Amount  #KPI

AverageTransAmt = transaction['Amount'].mean().astype('int64')
AverageTransAmt
```

Out[48]:
```
np.int64(18938)
```

In [49]:
```
transaction.head(5)
```

Out[49]:

	TID	Customer ID	Transaction Type	Transaction Mode	Amount
0	TRXN001	IND9933	Debit	UPI	7910
1	TRXN002	IND9328	Debit	UPI	2436
2	TRXN003	IND9324	Debit	UPI	9876
3	TRXN004	IND704	Debit	UPI	7063
4	TRXN005	IND2268	Debit	UPI	7016

Transaction Behavior – Key Insights :

Transaction activity varies widely among customers, with a small group driving the majority of transactions.

High transaction frequency often correlates with higher account balances and stronger customer engagement.

Customers with frequent transactions demonstrate consistent banking usage, indicating operational reliability and potential for upselling digital banking services.

---

5. Investment EDA :

In [50]:
```
investment.head()
```

Out[50]:

	Investment ID	Customer ID	Invest Monitor	Avenue	Investment Objective	Source
0	INV01	IND3536	Weekly	Public Provident Fund	Growth	Financial Consultants
1	INV02	IND8241	Weekly	Public Provident Fund	Growth	Financial Consultants
2	INV03	IND9583	Weekly	Public Provident Fund	Growth	Financial Consultants
3	INV04	IND9063	Weekly	Public Provident Fund	Growth	Financial Consultants
4	INV05	IND2343	Weekly	Public Provident Fund	Growth	Financial Consultants

In [51]:
```
investment.dtypes
```

Out[51]:
```
Investment ID           object
Customer ID             object
Invest Monitor          object
Avenue                  object
Investment Objective    object
Source                  object
dtype: object
```

In [52]:
```
investment.isna().sum()
```

Out[52]:
```
Investment ID           0
Customer ID             0
Invest Monitor          0
Avenue                  0
Investment Objective    0
Source                  0
dtype: int64
```

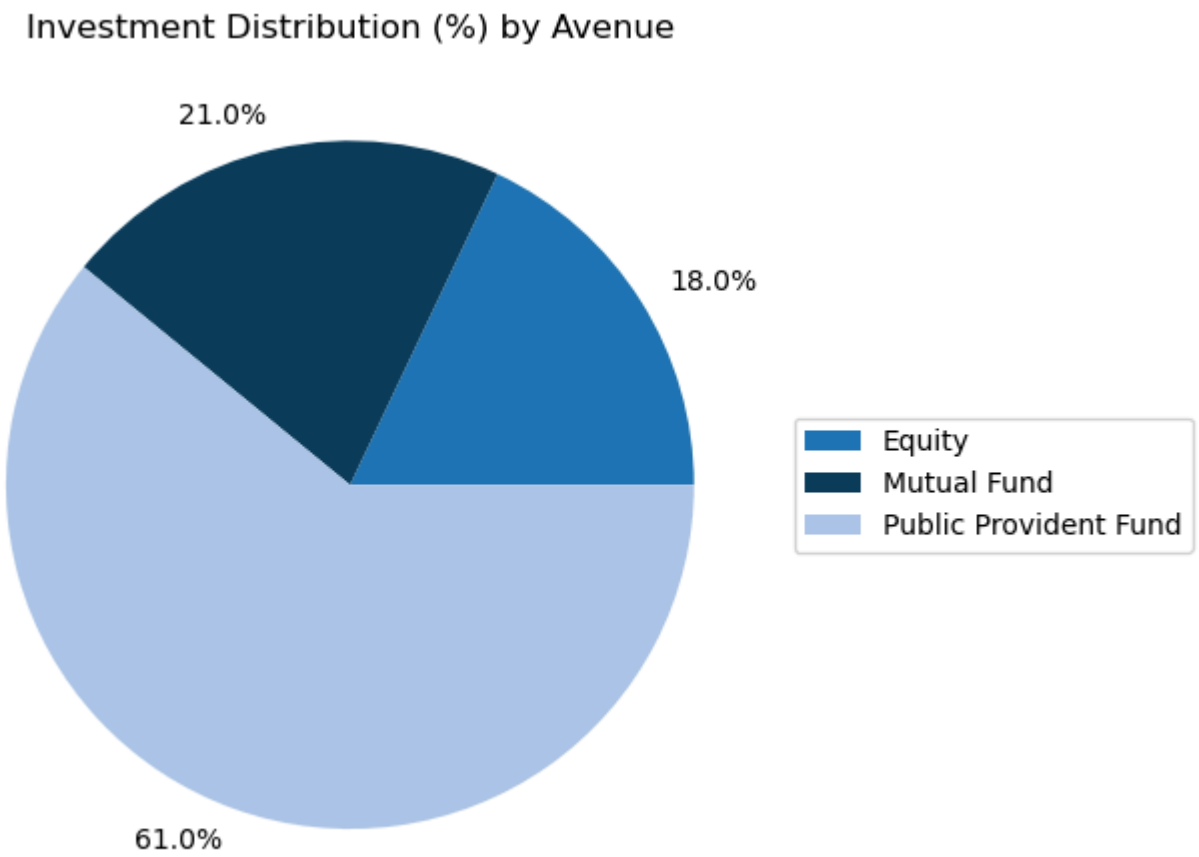
In [53]:
```
investment.shape
```

Out[53]:
```
(12000, 6)
```

In [54]:
```
# Total Investments  #KPI

TotalInvest = investment['Investment ID'].count()
TotalInvest
```

Out[54]:
```
np.int64(12000)
```

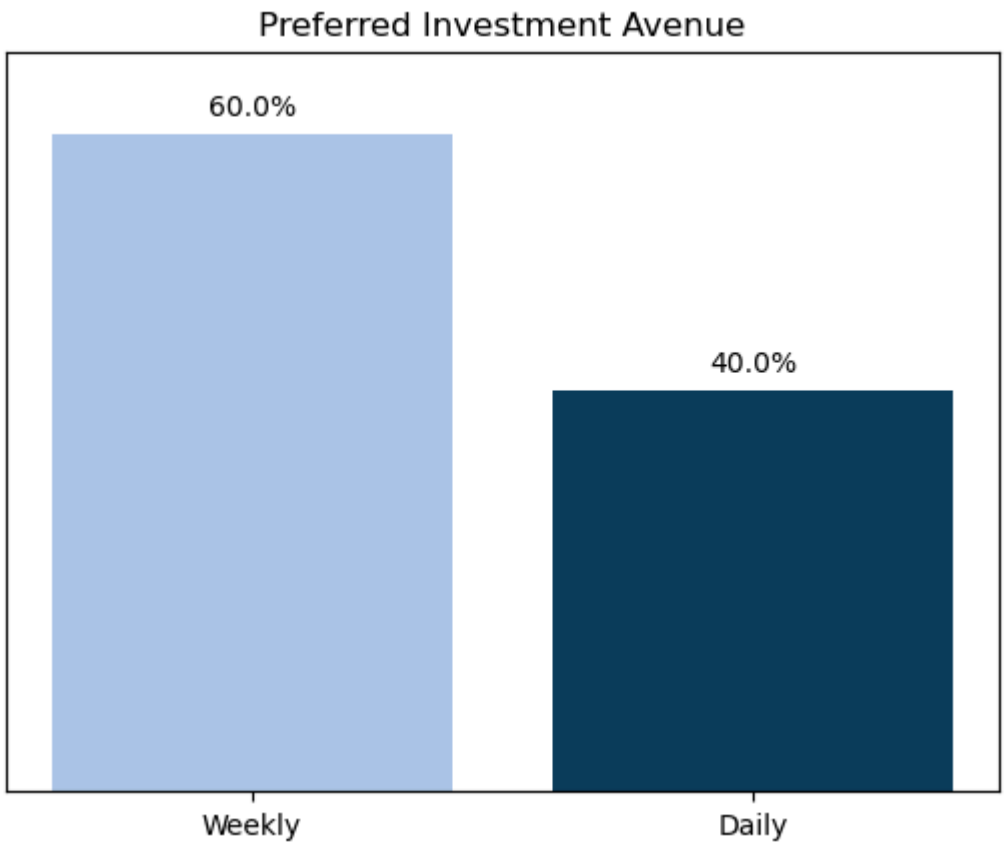
In [55]:
```
# Investments % Per Avenue

InvestXAvenue = investment.groupby(by=['Avenue']).agg({'Investment ID' : 'count'}).rename(columns={'Investment ID' : 'Total Investments'}).
InvestXAvenue

label = (InvestXAvenue['Total Investments'] / InvestXAvenue['Total Investments'].sum() * 100).round(0).astype(str) + '%'
color = ['#1f77b4', '#0b3c5d', '#aec7e8']

plt.pie(InvestXAvenue['Total Investments'], labels=label, colors=color)
plt.legend(InvestXAvenue['Avenue'], loc='center left',  bbox_to_anchor=(1, 0.5))
plt.title('Investment Distribution (%) by Avenue')
plt.tight_layout()
plt.show()
```



In [56]:

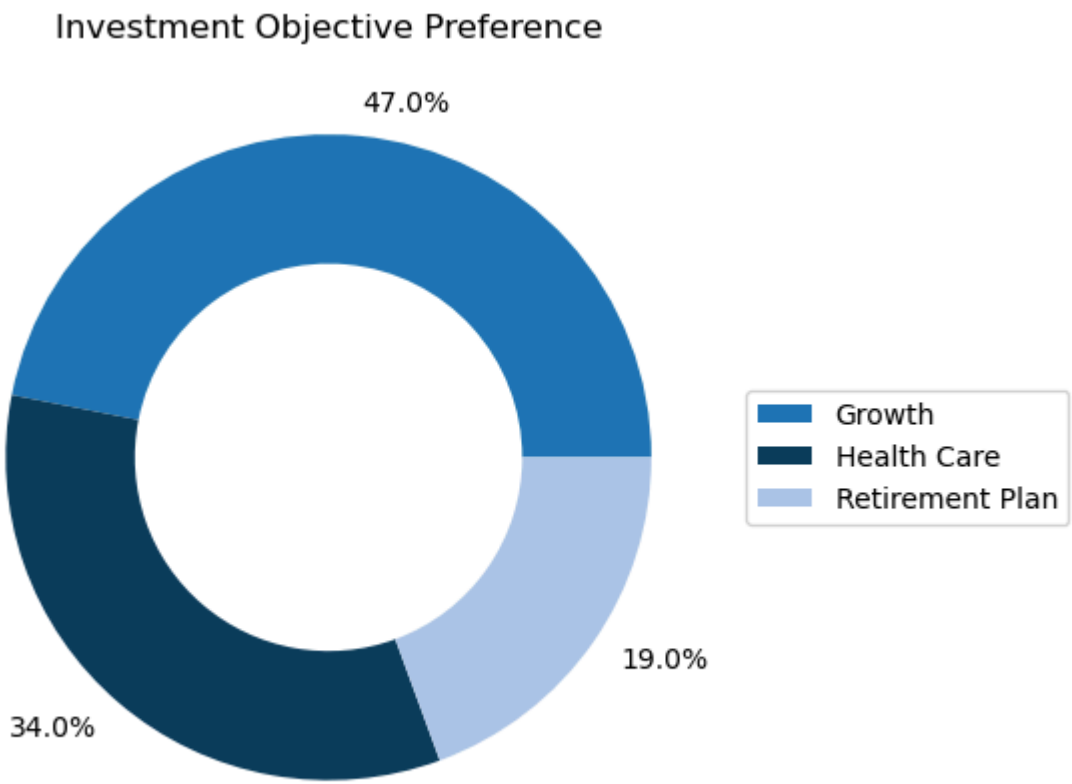
```
# Prefered Investment Monitor

InvestMonitor = investment.groupby(by=['Invest Monitor']).agg({'Investment ID' : 'count'}).rename(columns={'Investment ID' : 'Total Investm
InvestMonitor

color = ['#0b3c5d', '#aec7e8']
label = (InvestMonitor['Total Investments'] / InvestMonitor['Total Investments'].sum() * 100).round(0).astype(str) + '%'

bar = plt.bar(InvestMonitor['Invest Monitor'], InvestMonitor['Total Investments'], color=color)
plt.gca().invert_xaxis()
plt.title('Preferred Investment Avenue')
plt.bar_label(bar, labels=label, label_type='edge', padding=5)
plt.ylim([1000,8000])
plt.gca().get_yaxis().set_visible(False)
```

```
plt.show()
```



In [57]:

```
# Investment Objective Pattern

InvestXObj = investment.groupby(by=['Investment Objective']).agg({'Investment ID' : 'count'}).rename(columns={'Investment ID' : 'Total Inve
InvestXObj

label = (InvestXObj['Total Investments'] / InvestXObj['Total Investments'].sum() * 100).round(0).astype(str) + '%'
color = ['#1f77b4', '#0b3c5d', '#aec7e8']

plt.figure(figsize=(6,6))
plt.pie(InvestXObj['Total Investments'], labels=label, wedgeprops={'width' : 0.4}, colors=color)
plt.legend(InvestXObj['Investment Objective'], loc='center left', bbox_to_anchor=(1, 0.5))
plt.title('Investment Objective Preference')
plt.tight_layout()
plt.show()
```



In [58]:
```
# Prefered Source for Investment

SourceXInvest = investment.groupby(by=['Source']).agg({'Investment ID' : 'count'}).rename(columns={'Investment ID' : 'Total Investments'}).
SourceXInvest
```

Out[58]:

	Source	Total Investments
0	Financial Consultants	5635
1	Newspaper	2023
2	Social Media	4342

In [59]:
```
# Avenue preference by monitor type
```

```
InvestXAvenueXMonitor = investment.groupby(by=['Avenue', 'Invest Monitor']).agg({'Investment ID' : 'count'}).rename(columns={'Investment ID
InvestXAvenueXMonitor
```

Out[59]:

	Avenue	Invest Monitor	Total Investments
0	Equity	Daily	1103
1	Equity	Weekly	1050
2	Mutual Fund	Daily	1260
3	Mutual Fund	Weekly	1277
4	Public Provident Fund	Daily	2421
5	Public Provident Fund	Weekly	4889

Investment Analysis – Key Insights :

Investment behavior is concentrated in specific instruments, indicating customer preference for familiar and lower-risk investment options.

High-value investments are limited to a small customer segment, highlighting unequal wealth distribution.

Customers categorized under lower risk profiles tend to hold higher and more diversified investments, reinforcing the relationship between financial stability and investment behavior.

---

6. Cross-Table EDA :

In [60]:
```
joineddf = customer.merge(account, on='Customer ID').merge(loan, on='Customer ID', how='left').merge(transaction, on='Customer ID').merge(i
```

In [61]:
```
joineddf.head()
```

Out[61]:

	Customer ID	Customer Name	Gender	Age	Bank Name	Occupation	Income	Credit Cards Owened	Properties Owned	Age Group	...	Loan Type	TID	Transaction Type	Transaction Mode	Amou
0	IND001	Aarav Patil	Male	61	SBI	Loan Officer	1250340	3	1	Senior	...	Personal Loan	TRXN086	Debit	UPI	85
1	IND001	Aarav Patil	Male	61	SBI	Loan Officer	1250340	3	1	Senior	...	Personal Loan	TRXN238	Debit	UPI	27
2	IND001	Aarav Patil	Male	61	SBI	Loan Officer	1250340	3	1	Senior	...	Personal Loan	TRXN279	Debit	UPI	12
3	IND001	Aarav Patil	Male	61	SBI	Loan Officer	1250340	3	1	Senior	...	Personal Loan	TRXN356	Debit	UPI	11
4	IND001	Aarav Patil	Male	61	SBI	Loan Officer	1250340	3	1	Senior	...	Personal Loan	TRXN412	Debit	UPI	38

5 rows × 36 columns

In [62]:
```
joineddf.columns
```

Out[62]:
```
Index(['Customer ID', 'Customer Name', 'Gender', 'Age', 'Bank Name',
       'Occupation', 'Income', 'Credit Cards Owened', 'Properties Owned',
       'Age Group', 'Salary', 'Bank Loan', 'Bank Loan Status',
       'Business Lending', 'Business Lending Status', 'Current Account',
       'Savings Account', 'Fixed Deposit', 'Credit Card Balance',
       'Superannuation Savings', 'Joined Bank', 'Tendency', 'Risk Category',
       'Joined Year', 'Joined Month', 'Loan ID', 'Loan Type', 'TID',
       'Transaction Type', 'Transaction Mode', 'Amount', 'Investment ID',
       'Invest Monitor', 'Avenue', 'Investment Objective', 'Source'],
      dtype='object')
```

In [63]:
```
# Age Group wise saving tendency

AgeGrupXTendency = joineddf.groupby(by=['Age Group', 'Tendency'], observed=True).agg({'Customer ID' : 'count'}).rename(columns={'Customer I
AgeGrupXTendency
```

Out[63]:

	Age Group	Tendency	Customer Count
0	Young Adult	Neutral	9461
1	Young Adult	Saver	11238
2	Young Adult	Spender	7543
3	Adult	Neutral	15386
4	Adult	Saver	18961
5	Adult	Spender	14075
6	Senior	Neutral	6562
7	Senior	Saver	8576
8	Senior	Spender	5892

In [64]:

```
# Average Customer Salary By Age Group

AvgSalXAgeGrup = joineddf.groupby(by=['Age Group'], observed=True).agg({'Salary' : 'mean', 'Customer ID' : 'count'}).astype('int64').rename
AvgSalXAgeGrup
```

Out[64]:

	Age Group	Avg. Salary	Total Customers
0	Young Adult	82407	28242
1	Adult	83034	48422
2	Senior	84049	21030

In [65]:

```
# joineddf.to_excel('Indian_bank_cleaned_data.xlsx', index=False)
```

Conclusion :

The exploratory data analysis reveals that while the bank maintains a stable customer base, financial performance and risk exposure are highly concentrated within specific customer segments. High-risk customers contribute disproportionately to loan exposure, whereas low-risk customers demonstrate stronger investment behavior and transactional stability. These insights support the need for targeted risk management, personalized product strategies, and customer-centric decision-making.